

CYBERSECURITY HOMELAB PROJECT

Local Splunk SIEM Deployment

Linux, Active Directory, and Windows Endpoint Log Integration

A virtualized enterprise-style monitoring environment built with Proxmox, Splunk Enterprise, Windows Server, Active Directory, and Universal Forwarders.

Prepared by	Completion date	Project status
Matthew Hyman	August 29, 2026	Operational and validated

Core technologies

Splunk Enterprise 10.4.2 | Proxmox VE | Universal Forwarders | Active Directory | Linux | TCP/9997

Project Overview

This project expanded an existing Proxmox homelab into a functional local security monitoring environment. Splunk Enterprise was deployed on an Ubuntu virtual machine, configured to receive forwarded data, and connected to both Linux and Windows endpoints. A Windows domain controller and a separate domain-joined Windows client now forward operating-system and security events across an isolated lab network.

The final environment demonstrates more than a basic Splunk installation. It combines virtualization, network segmentation, Active Directory, endpoint configuration, time synchronization, event collection, SIEM searching, and systematic troubleshooting in one reproducible project.

Outcome: Ubuntu Server, DC01, and the Windows client are all forwarding telemetry successfully. Linux authentication events, Active Directory Security events including failed-login Event ID 4625, and more than 1,300 Windows client events were verified in Splunk.

Objectives

- Deploy a local Splunk Enterprise server inside Proxmox rather than relying on a cloud instance.
- Collect Windows Application, Security, and System event channels from domain endpoints.
- Forward logs over the isolated 10.10.10.0/24 network to TCP port 9997.
- Validate connectivity, service health, timestamps, and searchable security events.
- Preserve the completed environment as a stable homelab and portfolio milestone.

Lab Architecture

Component	Address / role	Function
Proxmox VE host	Hypervisor	Hosts the Ubuntu, Splunk, DC01, and Windows client virtual machines.
Splunk Server	10.0.0.190 / 10.10.10.30	Runs Splunk Enterprise and receives forwarded events on TCP 9997.
DC01	10.10.10.10	Provides mattlab.local Active Directory, DNS, domain time, and Windows Security telemetry.
Windows Client	10.10.10.20	Domain-joined endpoint forwarding Application, Security, and System logs.
Ubuntu Server	10.0.0.181	Runs the Linux Universal Forwarder and sends /var/log/auth.log telemetry.

Virtualized Lab Milestone

The environment was preserved with powered-off Proxmox snapshots after the Ubuntu, Splunk, and Windows AD integrations were validated. These snapshots provide convenient rollback points for the lab, while external backups remain a future improvement.

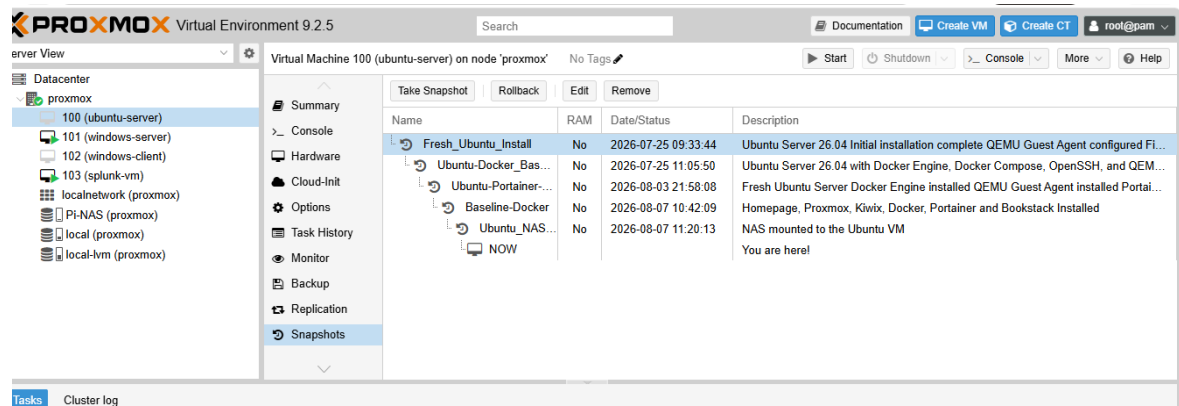


Figure 1. Proxmox virtual-machine inventory and powered-off Ubuntu milestone snapshots.

Splunk Enterprise Server

Splunk Enterprise 10.4.2 was deployed on a dedicated Ubuntu virtual machine inside Proxmox. The server is reachable at 10.0.0.190 for management and Splunk Web on TCP/8000. A second interface at 10.10.10.30 connects the SIEM to the isolated Windows lab without placing those systems directly on the household LAN.

The receiving service was enabled on TCP/9997 for Universal Forwarder traffic. This receiver became the common destination for the Ubuntu forwarder on the management network and both Windows forwarders on the isolated lab network.

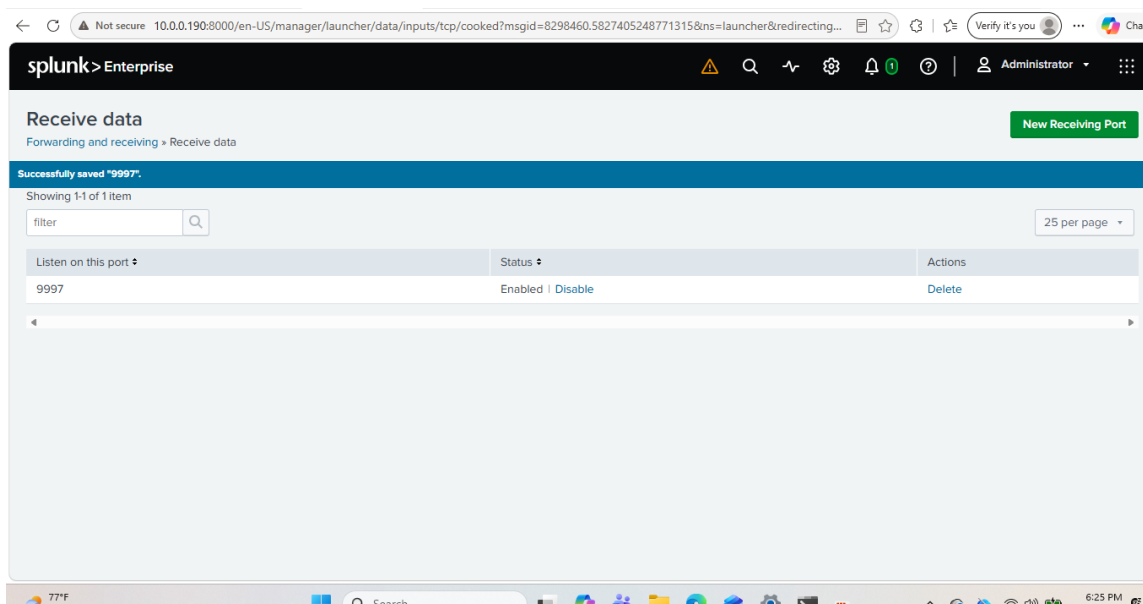


Figure 2. Splunk receiving configuration showing TCP port 9997 enabled.

Server Role and Startup

Splunk was configured to run as a non-root service and to start with the operating system. Keeping the SIEM on its own VM separated search, indexing, and receiver duties from the monitored Ubuntu application server.

Ubuntu Universal Forwarder Integration

The first monitored endpoint was Ubuntu Server at 10.0.0.181. Splunk Universal Forwarder 10.4.2 was installed from the Linux AMD64 package and configured to run under the dedicated splunkfwd account instead of root.

```
Options marked [*] produce a lot of output - pipe it through 'less' or 'more' !
matthew@ubuntu-server:~$ sudo dpkg -i splunkforwarder-10.4.2-linux-amd64.deb
Selecting previously unselected package splunkforwarder.
(Reading database... 139881 files and directories currently installed.)
Preparing to unpack splunkforwarder-10.4.2-linux-amd64.deb...
verify that this system has all the commands we will require to perform the preflight step
no need to run the splunk-preinstall upgrade check
Unpacking splunkforwarder (10.4.2)...
Setting up splunkforwarder (10.4.2)...
find: '/opt/splunkforwarder/lib/python3.7/site-packages': No such file or directory
find: '/opt/splunkforwarder/lib/python3.9/site-packages': No such file or directory
complete
matthew@ubuntu-server:~$ ls
bookstack  homepage  kiwix  splunkforwarder-10.4.2-linux-amd64.deb  uptime-kuma
matthew@ubuntu-server:~$ cd /opt
matthew@ubuntu-server:/opt$ ls
containerd  splunkforwarder
matthew@ubuntu-server:/opt$ cd splunkforwarder
matthew@ubuntu-server:/opt/splunkforwarder$ ls
README-splunk.txt  copyright.txt  ftr  lib  openssl  share  swidtag
bin  etc  include  license-eula.txt  opt  splunkforwarder-10.4.2-33c3bf42cd73-linux-amd64-manifest  uf
matthew@ubuntu-server:/opt/splunkforwarder$ cd /bin
matthew@ubuntu-server:/bin$
matthew@ubuntu-server:/bin$ ls
3cpio  gnunl  ngettext  sqinfo
```

Figure 3. Splunk Universal Forwarder package installation on Ubuntu Server.

Authentication Log Collection

The forwarder was pointed to 10.0.0.190:9997 and configured to monitor /var/log/auth.log with the linux_secure sourcetype in the main index. Membership in the adm group provided the service account with the read access needed for Ubuntu authentication telemetry.

```
matthew@ubuntu-server:/bin$ sudo -u splunkfwd /opt/splunkforwarder/bin/splunk list forward-server
Warning: Attempting to revert the SPLUNK_HOME ownership
Warning: Executing "chown -R splunkfwd:splunkfwd /opt/splunkforwarder"
Your session is invalid. Please login.
Splunk username: matthew
Password:
Active forwards:
  10.0.0.190:9997
Configured but inactive forwards:
  None
matthew@ubuntu-server:/bin$ |
```

Figure 4. Ubuntu forwarder reporting 10.0.0.190:9997 as its active destination.

Linux Validation

Failed SSH authentication attempts were generated deliberately and located with a host-specific SPL search. Splunk returned 76 linux_secure events, demonstrating collection, forwarding, indexing, and search across the complete Linux pipeline.

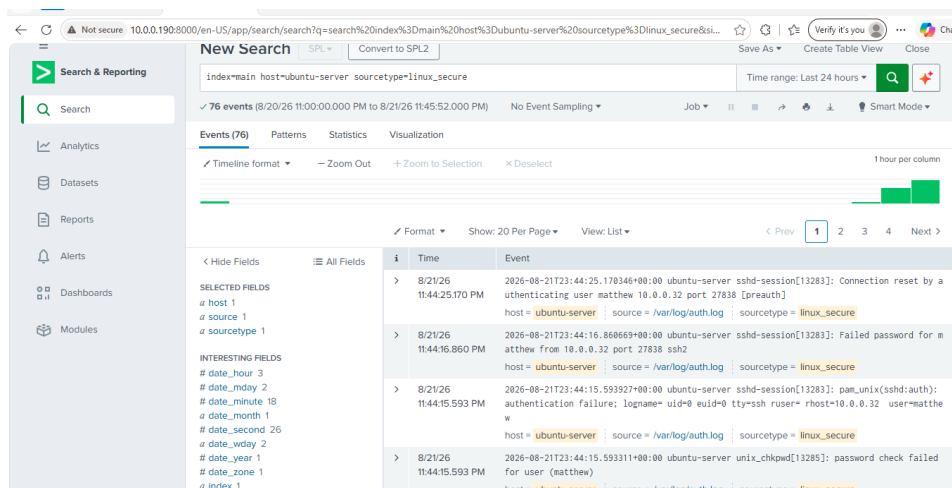


Figure 5. Ubuntu authentication events and failed SSH activity indexed in Splunk.

Windows Active Directory Integration

DC01 at 10.10.10.10 provides the mattlab.local domain, DNS, and domain time services. A Windows Splunk Universal Forwarder was installed on the controller and configured to send Application, Security, and System event channels to the Splunk receiver at 10.10.10.30:9997.

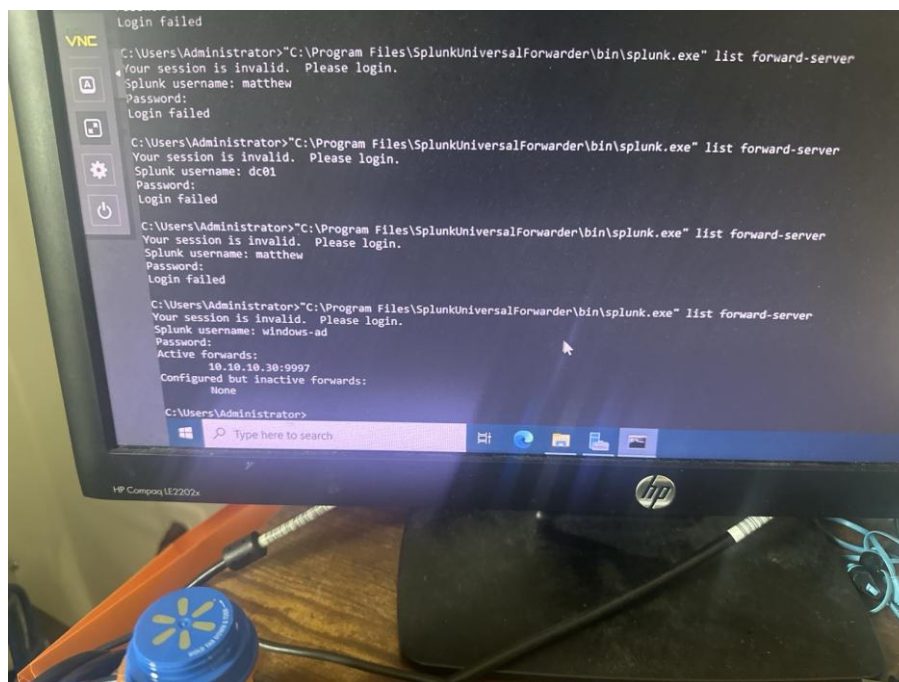


Figure 6. DC01 Universal Forwarder connected to 10.10.10.30:9997.

Time Synchronization Correction

During validation, DC01 was set to Pacific Time and referenced its Local CMOS Clock. That placed forwarded events in misleading time ranges. The controller was changed to Eastern Time and configured to synchronize with an external NTP source; domain clients now synchronize from DC01.mattlab.local.

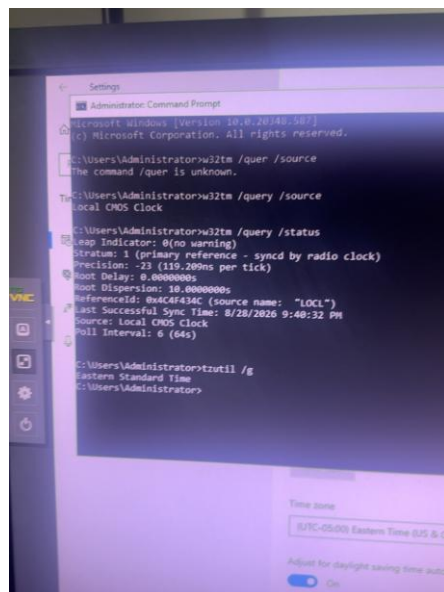


Figure 7. Diagnostic output identifying the original Local CMOS Clock source and timezone mismatch.

Active Directory Security Validation

Windows Security events were tested in layers. EventCode 4647 confirmed that Security-channel data was reaching Splunk. Failed-login Event ID 4625 was then generated locally on DC01, verified in the Windows event log, and finally located in Splunk after correcting the search time range and time synchronization.

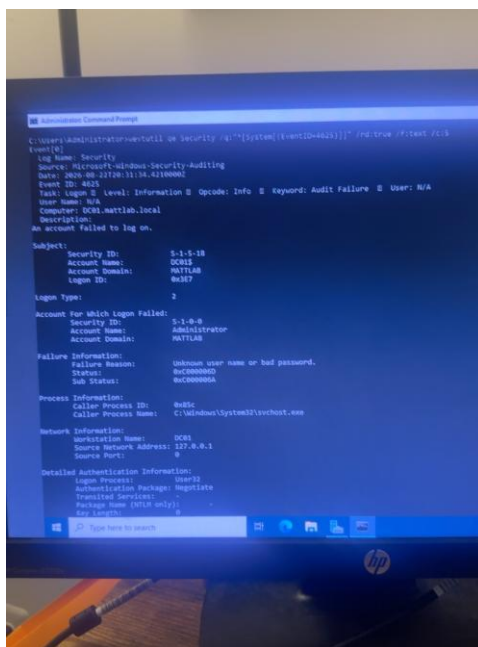


Figure 8. DC01 command-line Event Log query showing a locally generated Event ID 4625.

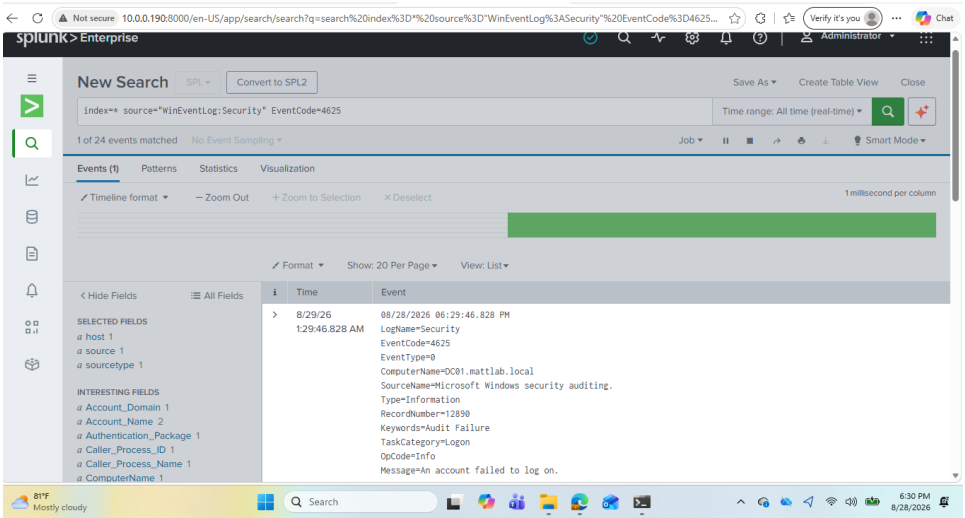


Figure 9. Splunk search confirming failed-login Event ID 4625 from DC01.mattlab.local.

Windows Client Integration

Network and Domain Validation

The domain-joined Windows client was assigned 10.10.10.20/24 with DC01 at 10.10.10.10 as its DNS server. The isolated adapter has no default gateway. ICMP tests to DC01 and the Splunk server succeeded, followed by a successful TCP connectivity test to port 9997.

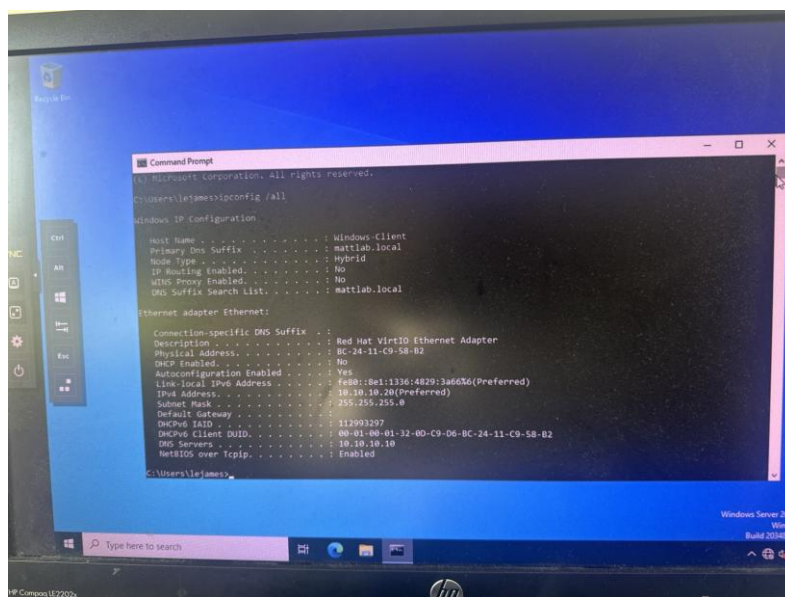


Figure 10. Windows client network configuration showing mattlab.local, 10.10.10.20/24, and DC01 as DNS.

Forwarder Deployment

Because the client cannot access the internet, the Universal Forwarder MSI was copied from DC01 through the protected administrative C\$ share. It was installed for an on-premises Splunk Enterprise instance, the unused deployment-server field was skipped, and the receiving indexer was set to 10.10.10.30:9997.

Windows Client Event Inputs

The installer established transport but did not create a local inputs.conf file. A custom configuration was created under etc\system\local to enable Application, Security, and System. This demonstrated an important SIEM distinction: an active forward proves connectivity, but useful telemetry still requires explicit input configuration.

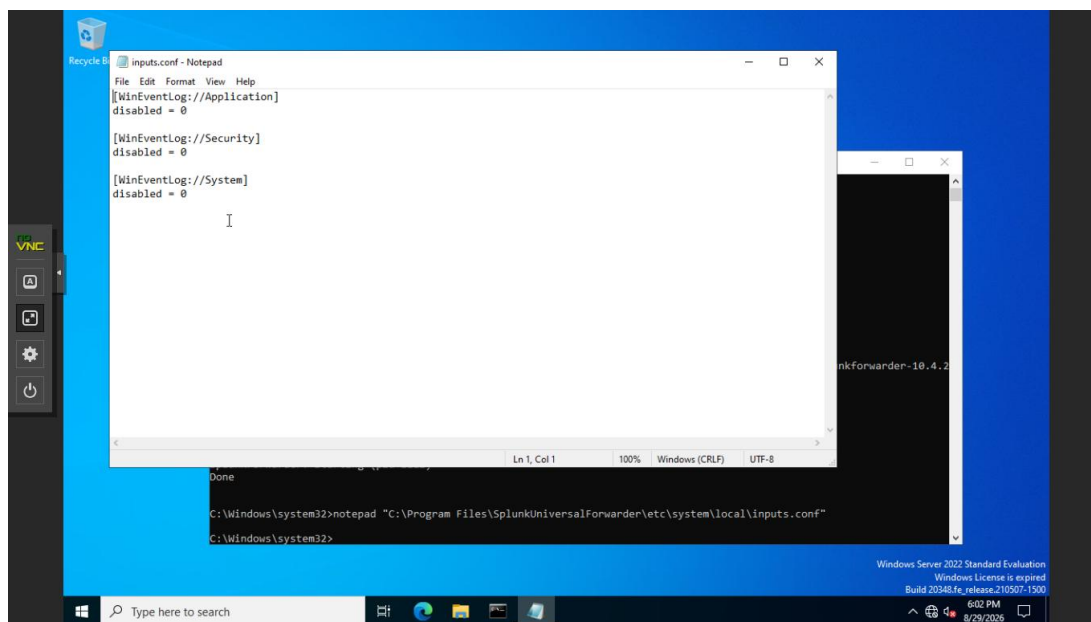


Figure 11. Windows client inputs.conf enabling Application, Security, and System event channels.

Service and Forwarding Verification

The SplunkForwarder Windows service reported RUNNING, and the CLI listed 10.10.10.30:9997 under Active forwards with no configured-but-inactive destinations.

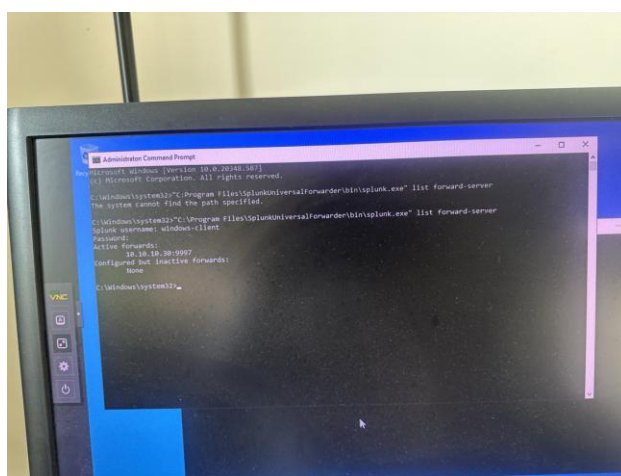


Figure 12. Windows client Universal Forwarder connected to 10.10.10.30:9997.

Cross-Platform Validation in Splunk

The completed Windows client integration produced 1,328 indexed events. Together with the Ubuntu linux_secure events and DC01 Security records, the searches confirmed that one local Splunk instance was collecting telemetry from Linux, an Active Directory domain controller, and a domain-joined endpoint.

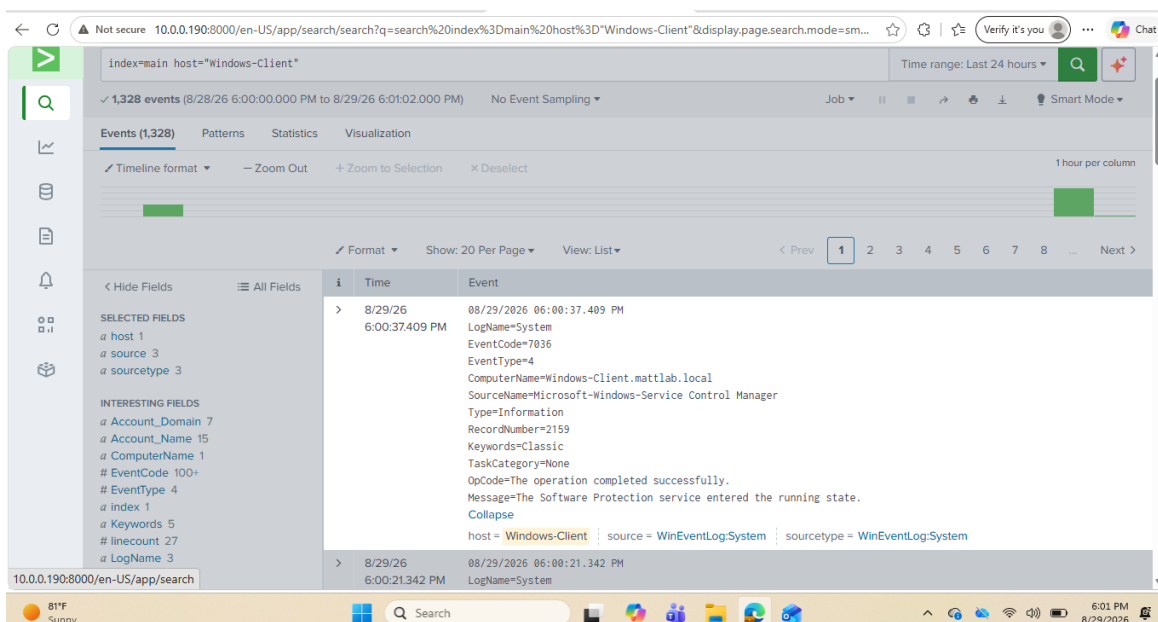


Figure 13. Splunk search results confirming 1,328 indexed events from Windows-Client.

Representative SPL Searches

The following searches were used across the three integrations:

```
index=main host=ubuntu-server sourcetype=linux_secure
index=* host=DC01 source="WinEventLog:Security"
index=* source="WinEventLog:Security" EventCode=4625
index=main host="Windows-Client" | stats count by source
```

Acceptance Results

Validation item	Result
Splunk Enterprise receiving service	Passed - TCP/9997 enabled
Ubuntu forwarder destination	Passed - 10.0.0.190:9997 active
Ubuntu authentication telemetry	Passed - 76 linux_secure events observed
DC01 forwarder destination	Passed - 10.10.10.30:9997 active
DC01 Security event collection	Passed - Event IDs 4647 and 4625 observed
Windows client to DC01 connectivity	Passed - 0% packet loss

Validation item	Result
Windows client to Splunk connectivity	Passed - 0% packet loss
Windows client receiver TCP/9997	Passed - TcpTestSucceeded: True
Windows client SplunkForwarder service	Passed - RUNNING
Windows client event inputs	Passed - Application, Security, and System enabled
Searchable Windows client data	Passed - 1,328 events observed

Troubleshooting and Analysis

The most valuable part of the project was diagnosing a condition that initially appeared to be failed log forwarding. Multiple layers were tested independently rather than assuming that a single successful check proved the entire pipeline.

Observed issue	Cause and resolution
The Ubuntu forwarder could not initially read auth.log.	The service account was placed in the adm group and the forwarder was restarted under splunkfwd.
Receiver and deployment-server roles were confused.	The endpoint was pointed to the Splunk receiving indexer on TCP/9997; the unused deployment-server field was left blank.
Expected events did not appear in Splunk searches.	The forwarding connection was active, but search syntax, time range selection, and incorrect VM time placement obscured the data. Searches were corrected and historical time ranges were used.
DC01 timestamps appeared inconsistent.	The controller used Pacific Time and Local CMOS Clock. It was moved to Eastern Time and configured for external NTP; the client now synchronizes from DC01.
A connected forwarder could still send no useful logs.	Transport and collection are separate layers. The client lacked inputs.conf, so Application, Security, and System stanzas were created explicitly.
The isolated client could not download the installer.	The MSI was reused from DC01 through the C\$ administrative share with domain Administrator credentials.
Forwarder CLI initially failed because of an incorrect path.	The installation directory and executable path were checked and the full quoted path was used.

Security and Operational Considerations

The design intentionally separates the lab endpoints from the normal household LAN through an isolated Proxmox bridge. Administrative shares required explicit credentials, and the Universal Forwarder runs as a managed Windows service. Private RFC1918 addresses are used throughout the environment.

One known hardening item remains: Splunk reported certificate-validation warnings related to default certificates and disabled server-certificate verification. The lab is operational, but production-quality deployment would replace default certificates, enable validation, restrict firewall rules to required hosts and ports, and centrally manage forwarder configurations.

Skills Demonstrated

- Splunk Enterprise administration, SPL searching, receiver configuration, and ingestion validation
- Windows Server 2022, Active Directory, DNS, domain accounts, event logs, and Windows Time service
- Splunk Universal Forwarder installation and inputs.conf / outputs.conf troubleshooting
- Proxmox networking, isolated virtual bridges, static addressing, and multi-VM integration
- TCP/IP troubleshooting with ping, Test-NetConnection, service queries, and application logs
- Linux service management with systemd and cross-platform log collection
- Technical documentation, evidence capture, root-cause analysis, and milestone snapshots

Project Reflection

This project reinforced that “the connection works” is not the same as “the data pipeline works.” A reliable SIEM workflow depends on endpoint collection, forwarder service health, network transport, receiver configuration, time synchronization, indexing, and correct search logic. Testing each layer separately made it possible to find the real issues without repeatedly changing settings that were already correct.

The final environment now provides a stable base for future detection engineering. Additional Windows clients, Linux servers, applications, and simulated security events can be added without redesigning the entire architecture.

Future Enhancements

- Replace Splunk default certificates and enable server-certificate validation.
- Create dashboards and alerts for repeated failed logins, account lockouts, and privileged activity.
- Add a deployment server if the number of forwarders grows enough to justify centralized management.
- Restrict temporary interfaces and firewall exposure after administrative tasks are complete.
- Implement external Proxmox backups so snapshots are not the only rollback mechanism.